Autonomous Vehicle Intersection Management System (AVIMS)

Subject:	Special Emphasis A (Embedded System)
Team Members:	Md Wasib Kamal Nirjon
	Md Ratul Ahmed Alvi
	Moshiour Rahman Prince
	Md Tawhidur Rahman Tuhin
University:	Hamm-Lippstadt University of Applied Sciences

1. Introduction

The Autonomous Vehicle Intersection Management System (AVIMS) is a real-time, server-based framework that coordinates autonomous vehicles (AVs) at unsignalized intersections. Instead of conventional traffic signals, AVIMS uses V2V (Vehicle-to-Vehicle) and V2I (Vehicle-to-Infrastructure) communication to predict trajectories, detect conflicts, and issue crossing authorizations — all within strict timing constraints. The system is built around four key actors: the Autonomous Vehicle, Roadside Sensor, Primary Server, and Backup Server, forming a fault-tolerant, low-latency intersection pipeline.

2. Project Objectives

- Detect approaching vehicles via roadside sensors.
- Receive and process vehicle state data (position, speed, ETA) through V2V and V2I.
- Identify trajectory conflicts between simultaneously approaching vehicles.
- Generate safe crossing schedules within a 50 ms decision window.
- Grant or deny crossing authorization using MM/1 or MM/2 coordination.
- Ensure no vehicle enters without explicit server permission.
- Maintain reliability via primary–backup server synchronization.
- Monitor vehicle exit and update intersection state after each crossing.

3. System Methodology

3.1 FIFO Scheduling

Requests are queued and processed in arrival order, ensuring fair and deterministic scheduling with no starvation.

3.2 MM/1 Coordination — Single Vehicle Crossing

Used when conflicting trajectories are detected. Only one vehicle crosses at a time; all others receive wait/deny instructions until the intersection is clear.

3.3 MM/2 Coordination — Simultaneous Non-Conflicting Crossing

Used when two or more vehicles have non-overlapping paths. Multiple vehicles cross simultaneously, increasing throughput without compromising safety.

3.4 V2V Communication

Vehicles broadcast position and ETA to nearby vehicles directly, giving the server a complete picture of all approaching traffic.

3.5 V2I Communication

Each AV transmits its state to the roadside unit and Primary Server, which uses it for conflict detection, scheduling, and issuing permission or denial.

3.6 Why 50 ms? — Justification of the Real-Time Constraint

The 50 ms deadline is derived from three converging factors:

Kinematics

At 50 km/h (≈13.9 m/s) a vehicle travels ~0.7 m in 50 ms — well within the pre-entry detection zone. The server must decide before the vehicle reaches the point of no return where emergency braking alone cannot prevent intersection entry.

Communication Stack Budget

A V2I round trip over IEEE 802.11p (DSRC) or C-V2X completes in ~10–20 ms including stack processing. The 50 ms budget covers two full round trips plus server computation, with headroom for retransmission.

Standards Alignment & Hardware Feasibility

ETSI EN 302 637-2 mandates ≤100 ms for cooperative awareness messages; IEEE 1609.3 (WAVE) targets <50 ms for safety-critical V2X messages. On an ARM Cortex-A class roadside unit (RTOS), conflict detection and scheduling for up to 8 vehicles completes in <5 ms — giving a 10× safety margin within the budget.

> *In summary, 50 ms satisfies kinematic safety, communication latency, international standards, and embedded hardware constraints simultaneously.*

4. UML Diagrams

4.1 Requirements Diagram

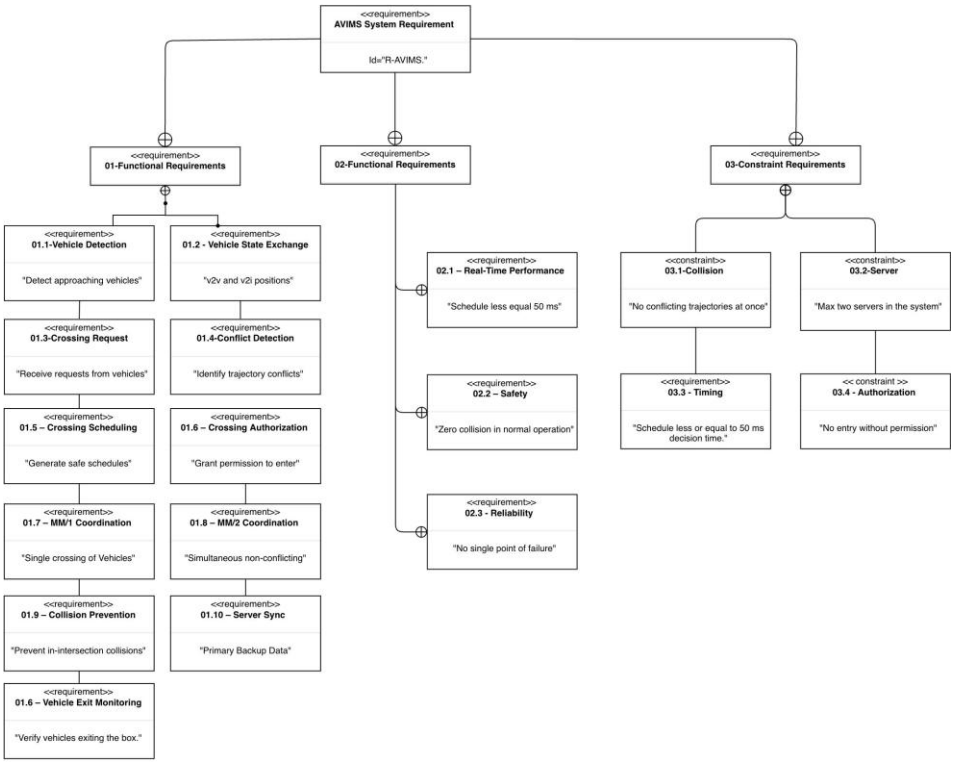

Figure 1 – AVIMS Requirements Diagram

Overview

The Requirements Diagram organises every system requirement into a three-branch hierarchy rooted at 'AVIMS System Requirement (Id=R-AVIMS.)'. Each branch is connected by a containment relationship (circled-plus symbol), ensuring full traceability from root to implementation.

Functional Requirements (01)

- **01.1 Vehicle Detection** — Detect approaching vehicles. → UC-01, Sequence step 1.
- **01.2 Vehicle State Exchange** — Exchange V2V and V2I position/ETA data. → UC-03, Seq. steps 1.1 & 2.
- **01.3 Crossing Request** — Receive formal crossing requests from AVs. → UC-04, Seq. step 2.1.
- **01.4 Conflict Detection** — Identify overlapping trajectories. → UC-06, Seq. step 2.1.1.
- **01.5 Crossing Scheduling** — Produce a safe schedule ≤50 ms. → UC-07, Seq. step 2.1.1.1.
- **01.6 Crossing Authorization** — Issue explicit crossing permission. → UC-08, Seq. steps 3 & 4.
- **01.7 MM/1 Coordination** — Single-vehicle crossing mode. → UC-09.
- **01.8 MM/2 Coordination** — Simultaneous non-conflicting crossing. → UC-10.
- **01.9 Collision Prevention** — Guarantee zero in-intersection collisions.
- **01.10 Server Sync** — Sync State to Back up after every decision. → UC-11, Seq. step 2.1.1.1.1.
- **01.6* Vehicle Exit Monitor** — Verify and record vehicle exit. → UC-13, Seq. step 6.

Non-Functional Requirements (02)

- **02.1 Real-Time Performance** — Scheduling decisions within ≤50 ms (see Section 3.6).
- **02.2 Safety** — Zero collisions during normal operation.
- **02.3 Reliability** — No single point of failure — satisfied by the primary–backup architecture.

Constraint Requirements (03)

- **03.1 Collision** — No two conflicting trajectories active simultaneously inside the intersection.
- **03.2 Server** — Maximum two servers in the system (Primary + Backup).
- **03.3 Timing** — Schedule must be generated within ≤50 ms of receiving a request.
- **03.4 Authorization** — No vehicle may enter without a valid server-issued permission.

Cross-Diagram Links

Every requirement map to a use case and a labelled sequence message (e.g., FR-05 + NFR-01 + C-03 all converge on the ≤50 ms schedule step). Constraints C-01 and C-04 appear as guards in the Sequence Diagram and as decision diamonds in the Activity Diagram.

4.2 Use Case Diagram

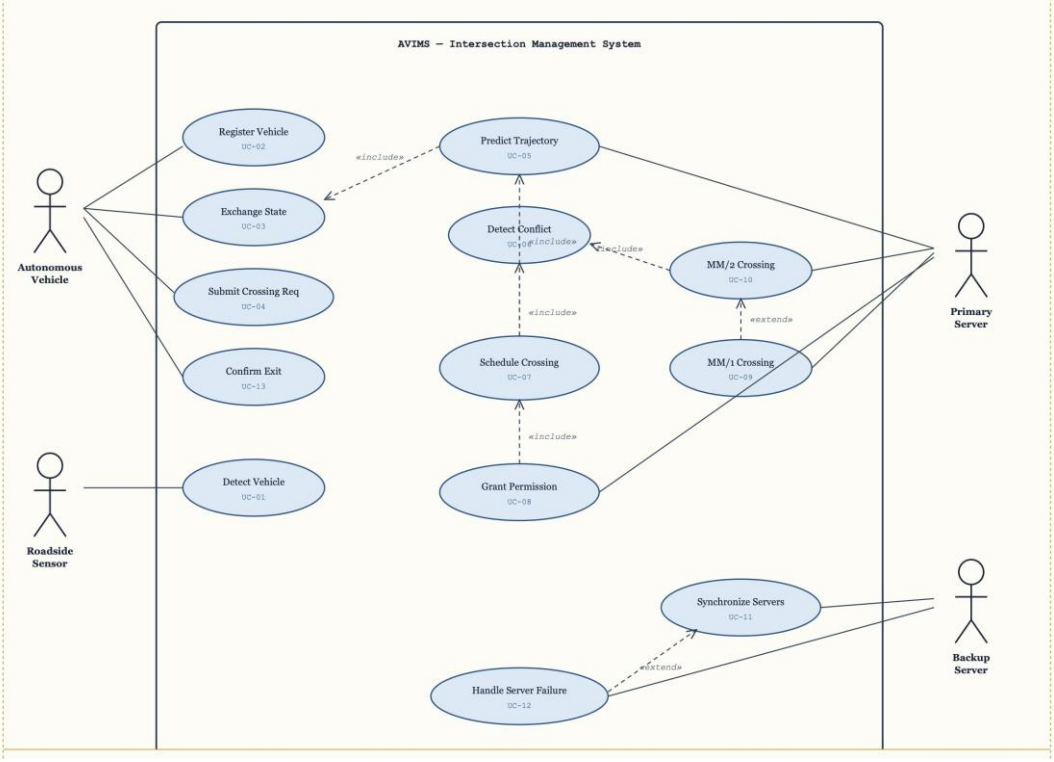

Figure 2 – AVIMS Use Case Diagram

Actors

- **Autonomous Vehicle** — Primary client. Registers, exchanges state, submits requests, and confirms exit (UC-02, 03, 04, 13).
- **Roadside Sensor** — Edge detector. Triggers the entire pipeline via Detect Vehicle (UC-01).
- **Primary Server** — Central coordinator. Drives trajectory prediction, conflict detection, scheduling, and permission (UC-05 to UC-10).
- **Backup Server** — Replication node. Handles server sync (UC-11) and failure recovery (UC-12).

Use Cases and Relationships

ID	Use Case	Actor(s)	Rel.	Description
UC-01	Detect Vehicle	Roadside Sensor	Assoc.	Entry trigger. Sensor detects AV in approach zone.
UC-02	Register Vehicle	AV	Assoc.	AV registers itself with the system on approach.
UC-03	Exchange State	AV	«incl.»→05	V2V + V2I state broadcast; always includes UC-05.
UC-04	Submit Crossing Req.	AV	Assoc.	Formal crossing request sent to Primary Server.
UC-05	Predict Trajectory	Server (internal)	Incl. by 03	Predicts future vehicle path from state data.
ID	**Use Case**	**Actor(s)**	**Rel.**	**Description**

UC-06	Detect Conflict	Server (internal)	«incl.»→05	Checks for geometric path overlaps; always uses UC-05.
UC-07	Schedule Crossing	Server (internal)	«incl.»→06	Generates ≤50 ms collision-free schedule; uses UC-06.
UC-08	Grant Permission	Primary Server	«incl.»→07	Issues crossing authorisation; always uses UC-07.
UC-09	MM/1 Crossing	AV	«ext.» UC-10	Extends UC-10 when only one vehicle can cross safely.
UC-10	MM/2 Crossing	Primary Server, AV	«incl.»→06	Base coordination; simultaneous non-conflicting crossing.
UC-11	Synchronize Servers	Primary + Backup	Assoc.	Replicates schedule and state to Backup after each decision.
UC-12	Handle Srv. Failure	Backup (conditional)	«ext.» UC-11	Extends UC-11 only when Primary Server becomes unreachable.
UC-13	Confirm Exit	AV	Assoc.	AV signals full exit; frees the intersection slot.

Include / Extend Chain

The «include» chain is mandatory and runs as:

UC-03 → *UC-05* → *UC-06* → *UC-07* → *UC-08*

Every state exchange leads to a schedule and permission — no orphaned use cases. «extend» relationships are conditional: UC-09 fires when only one vehicle can cross; UC-12 fires only when the Primary Server fails.

4.3 Activity Diagram

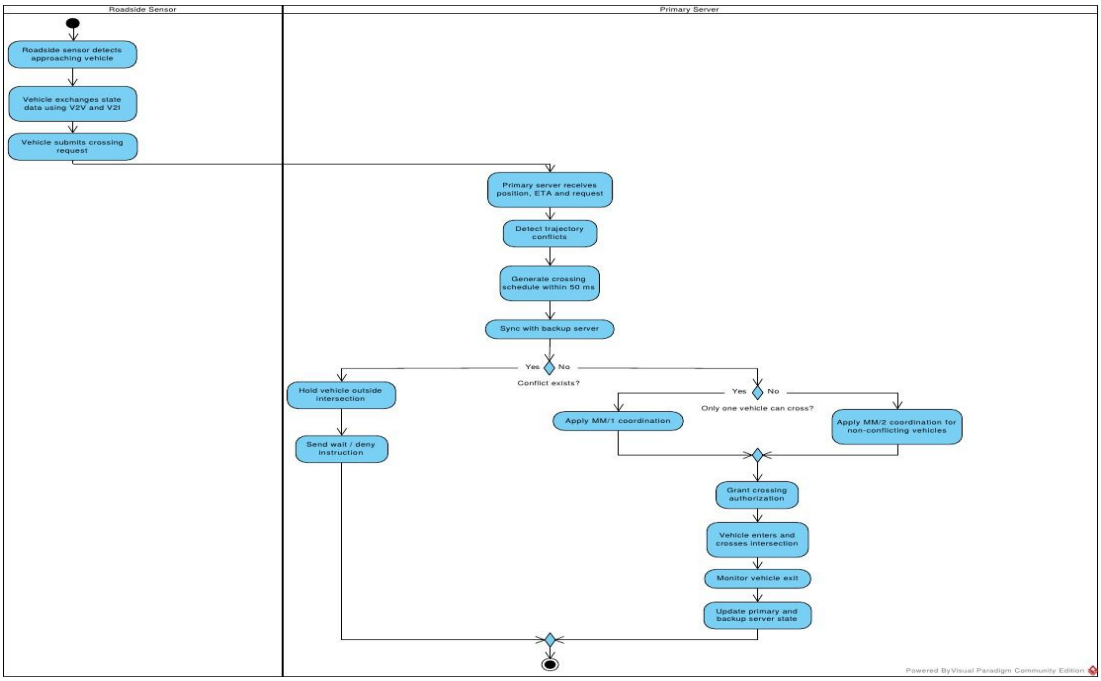

Figure 3 – AVIMS Activity Diagram

Overview

The Activity Diagram captures the complete end-to-end operational flow of AVIMS using two swim lanes — Roadside Sensor (left) and Primary Server (right) — and two decision diamonds that branch the flow based on conflict status and crossing mode.

Swim Lane 1 — Roadside Sensor

- **Detect approaching vehicle** — Sensor identifies AV in the detection zone. Trigger for the entire cycle (FR-01 / UC-01).
- **Exchange state via V2V and V2I** — AV broadcasts position, speed, and ETA to peers and infrastructure (FR-02 / UC-03).
- **Submit crossing request** — AV formally requests permission; handed off to Primary Server (FR-03 / UC-04).

Swim Lane 2 — Primary Server

- **Receive position, ETA and request** — Server collects all V2I state data as input for processing.
- **Detect trajectory conflicts** — Computes projected paths and checks for geometric overlaps (FR-04 / UC-06).
- **Generate schedule within 50 ms** — Produces a conflict-free crossing schedule within the timing constraint (NFR-01 / C-03).
- **Sync with Backup Server** — Immediately replicates schedule to Backup, satisfying reliability

requirement 02.3.

Decision 1 — Conflict Exists?

- Yes → Hold vehicle outside intersection → Send wait/deny instruction → End.
- No → Proceed to Decision 2.

Decision 2 — Only One Vehicle Can Cross?

- Yes → Apply MM/1 coordination (one vehicle crosses, others wait).
- No → Apply MM/2 coordination (multiple non-conflicting vehicles cross simultaneously).

Post-Authorization Steps

- **Grant crossing authorization** — Server issues a signed permission token to the authorised AV(s).
- **Vehicle enters and crosses intersection** — AV physically traverses the intersection.
- **Monitor vehicle exit** — Server tracks exit via sensor feedback.
- **Update primary and backup server state** — Both servers record the completed crossing and free the slot for the next cycle.

4.4 Sequence Diagram

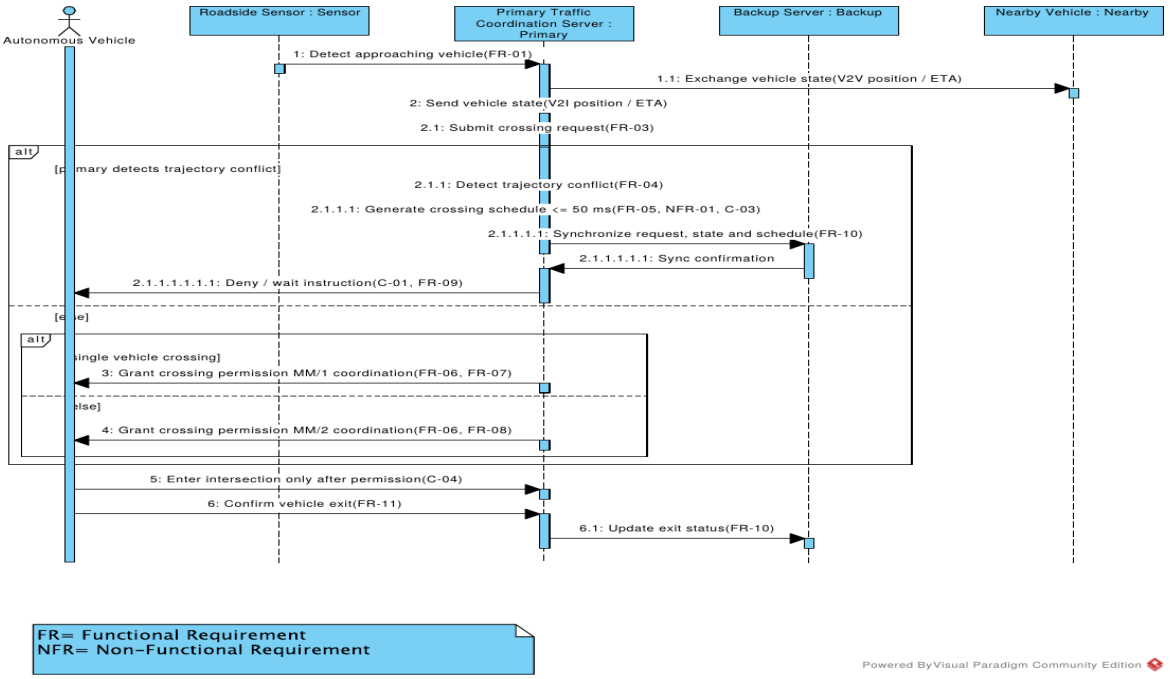

Figure 4 – AVIMS Sequence Diagram

Overview

The Sequence Diagram shows the time-ordered message exchange between five participants across a single intersection management cycle. Each message carries a requirement tag (FR / NFR / C) linking it to the Requirements Diagram.

Participants: Autonomous Vehicle · Roadside Sensor · Primary Server · Backup Server · Nearby Vehicle

Message Flow

Step	From → To	Requirement	Description
1	Sensor → Server	FR-01	Sensor detects AV and notifies Primary Server — entry trigger.
1.1	Server → Nearby Vehicle	V2V	Server relays state broadcast; nearby vehicles share position and ETA.

2	AV → Server	V2I	AV sends position, speed, heading, and ETA directly to server.
2.1	AV → Server	FR-03	AV submits formal crossing request, opening server-side processing.
2.1.1	Server (internal)	FR-04	[alt: conflict detected] Server computes trajectory overlaps.
Step	**From → To**	**Requirement**	**Description**
2.1.1.1	Server (internal)	FR-05, NFR-01, C-03	Server generates crossing schedule — must complete within ≤50 ms.
2.1.1.1.1	Server → Backup	FR-10	Full state (request + schedule) pushed to Backup Server immediately.
2.1.1.1.1.1	Backup → Server	—	Backup confirms sync receipt; fault tolerance is now active.
2.1.1.1.1.1.1	Server → AV	C-01, FR-09	[else branch] Conflict found — server sends deny/wait instruction.
3	Server → AV	FR-06, FR-07	[single vehicle] MM/1 permission granted to one AV only.
4	Server → AV	FR-06, FR-08	[else] MM/2 permission granted to multiple non-conflicting AVs.
5	Server → AV	C-04	Server confirms: AV may enter intersection only with this permission.
6	AV → Server	FR-11	AV confirms it has fully exited; intersection slot is freed.
6.1	Server → Backup	FR-10	Exit status propagated to Backup; both servers stay in sync.

Alt Fragments

• Outer alt [primary detects trajectory conflict] / [else] — If conflict detected: schedule → sync → deny/wait (steps 2.1.1 to 2.1.1.1.1.1.1). If no conflict: fall through to inner alt.

• Inner alt [single vehicle crossing] / [else] — One vehicle: MM/1 permission (step 3). Multiple non-conflicting vehicles: MM/2 permission (step 4).

Cross-Diagram Consistency

The outer alt guard mirrors Activity Diagram Decision 1 ('Conflict exists?'); the inner alt maps to Decision 2 ('Only one vehicle can cross?'). All FR/C tags link back to the Requirements Diagram, making the design fully traceable across all four UML models.

5. Conclusion

AVIMS delivers a traceable and technical sound design for autonomous intersection management. The Requirements Diagram defines a clear hierarchical contract; the Use Case Diagram maps every requirement to a system function; the Sequence Diagram specifies the exact inter-component message protocol; and the Activity Diagram narrates the complete operational cycle. The 50 ms scheduling constraint is justified by vehicle kinematics, V2X communication stack budgets, and ETSI/IEEE standards, and is readily achievable on modern embedded hardware. Together, the four models form a consistent, inter-linked specification ready for embedded system implementation.